Docker Multi-Stage Build Assignment

Java Maven WAR Build + Apache Tomcat Deployment

Objective

Build a Java web application using Maven in a builder stage, then copy only the generated WAR file into a lightweight Tomcat runtime image. Verify that the application runs through a mapped host port.

Prerequisites

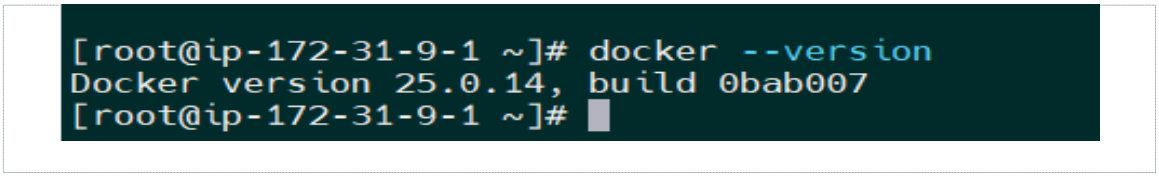
Docker installed and running; terminal access; internet access to pull Maven and Tomcat images. No local Maven or Java installation is required.

Step 1: Verify Docker

Run the following commands to check the Docker installation and daemon.

```
docker --version
docker info
```

Screenshot evidence: Docker version and Docker info output

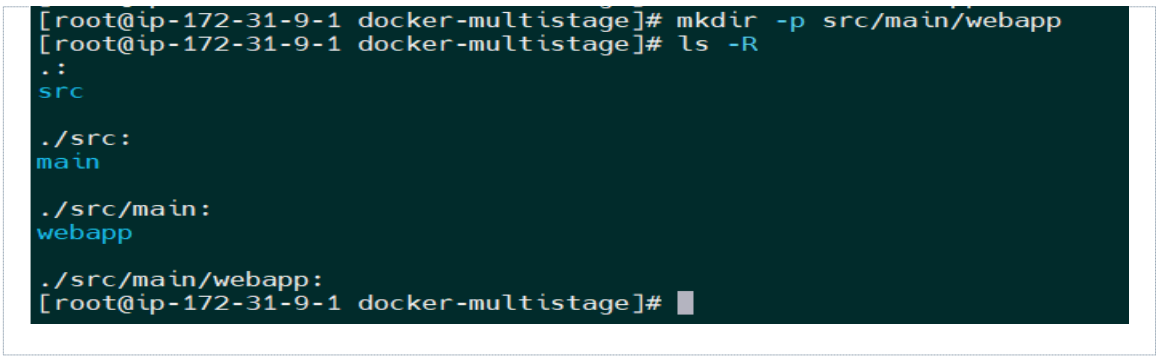


Step 2: Create the project folders

Create a working directory and the Maven webapp folder structure. Use PowerShell on Windows:

```
mkdir docker-multistage
cd docker-multistage
mkdir src\main\webapp
```

Screenshot evidence: Project folder structure

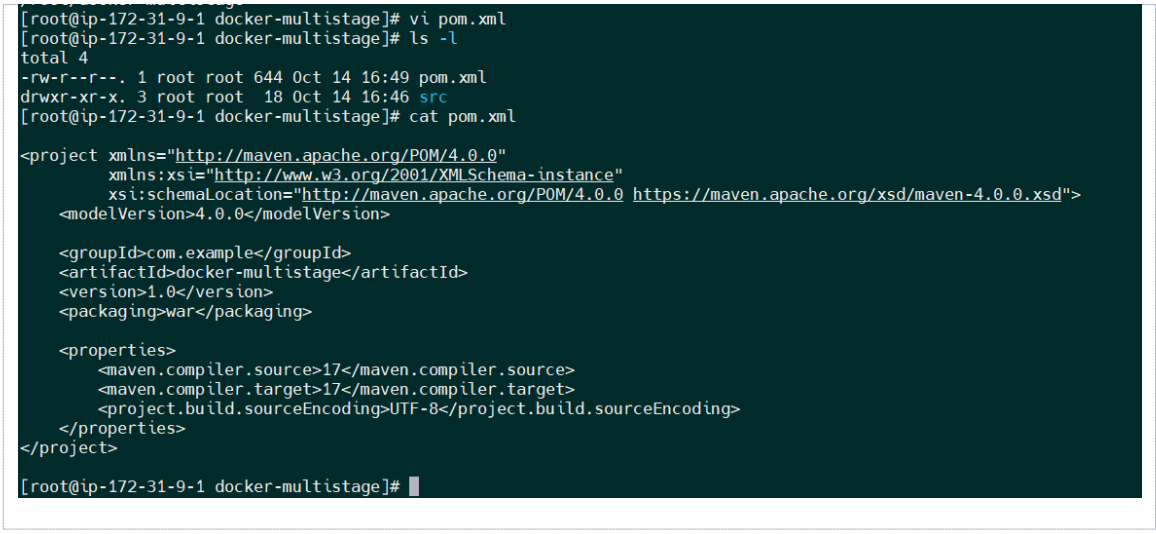


Step 3: Create pom.xml

Create a file named pom.xml in docker-multistage and paste the following XML:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>docker-demo</artifactId>
  <version>1.0.0</version>
  <packaging>war</packaging>
  <build>
    <finalName>docker-demo</finalName>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-war-plugin</artifactId>
        <version>3.4.0</version>
        <configuration>
          <failOnMissingWebXml>false</failOnMissingWebXml>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

Screenshot evidence: pom.xml saved in project root



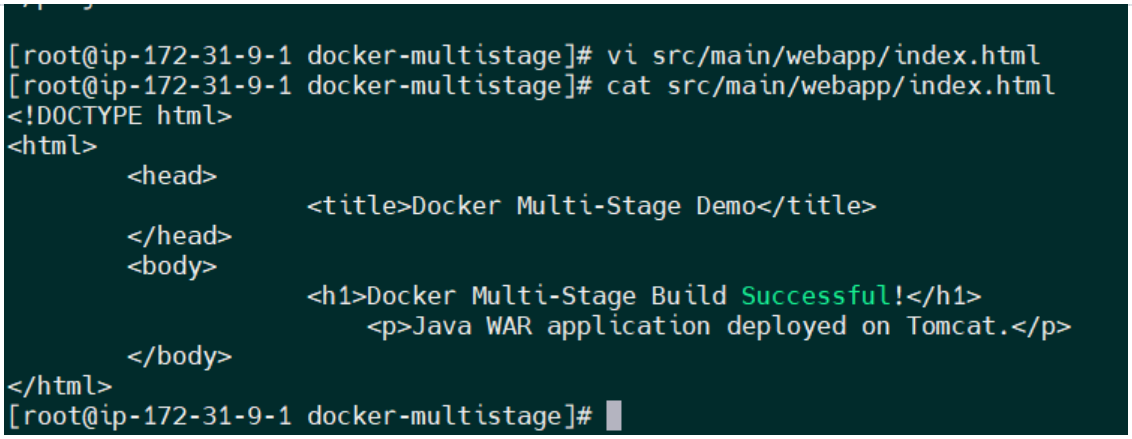
Step 4: Create the web application

Create src/main/webapp/index.html with the following content:

```
<!DOCTYPE html>
<html>
```

```
<head><title>Docker Multi-Stage Demo</title></head>
<body>
  <h1>Docker Multi-Stage Build Successful!</h1>
  <p>Java WAR application deployed on Tomcat.</p>
</body>
</html>
```

Screenshot evidence: index.html content

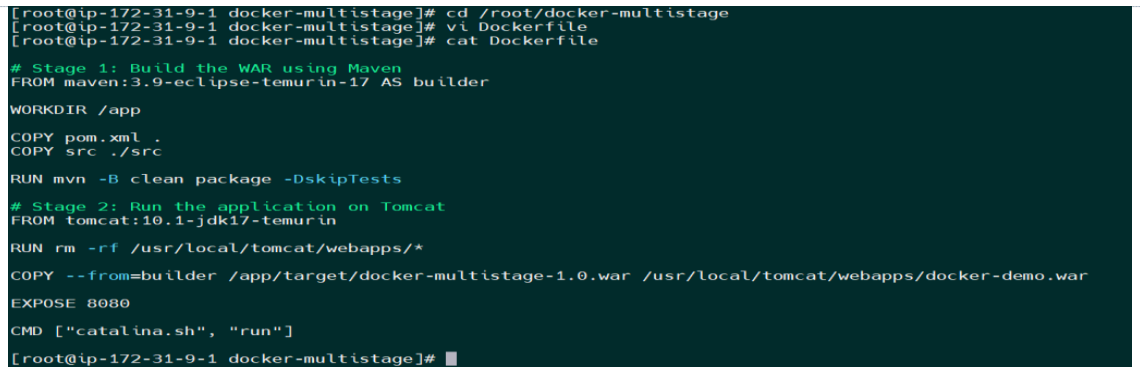


Step 5: Write the multi-stage Dockerfile

Create a file named Dockerfile (no extension) in the project root. The first stage compiles/packages the WAR using Maven; the second stage runs Tomcat and copies only the WAR.

```
# Stage 1: Build
FROM maven:3.9-eclipse-temurin-17 AS builder
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn -B clean package -DskipTests

# Stage 2: Runtime
FROM tomcat:10.1-jdk17-temurin
RUN rm -rf /usr/local/tomcat/webapps/*
COPY --from=builder /app/target/docker-demo.war \
     /usr/local/tomcat/webapps/docker-demo.war
EXPOSE 8080
CMD ["catalina.sh", "run"]
```

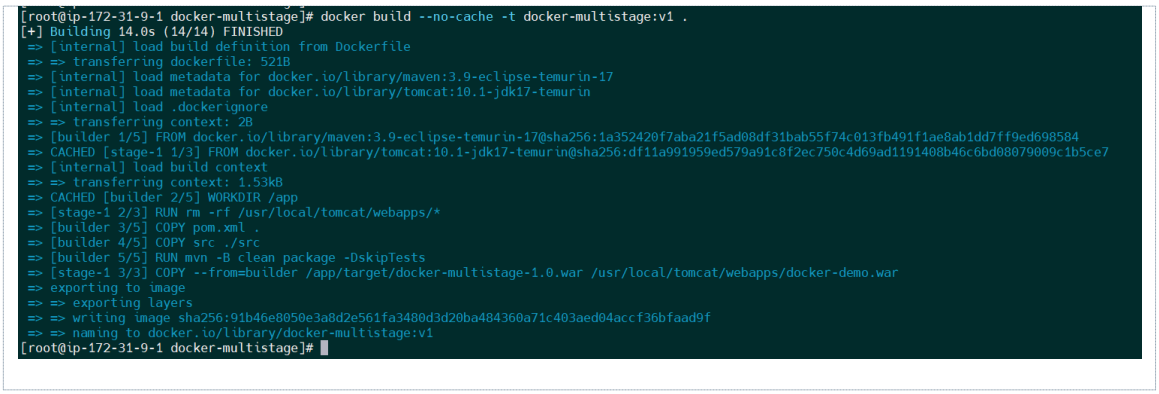


Step 6: Build the Docker image

From the docker-multistage directory (where Dockerfile exists), run:

```
docker build -t docker-multistage:v1 .
```

Screenshot evidence: Successful image build output

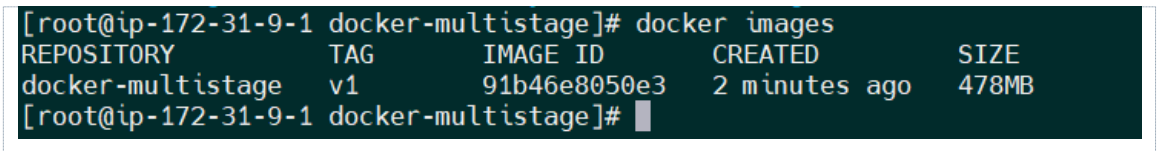


Step 7: Check the Docker image

Confirm the new image exists:

```
docker images
```

Screenshot evidence: docker-multistage:v1 image listed



Step 8: Run the container with port mapping

Map host port 8081 to Tomcat container port 8080:

```
docker run -d --name multistage-tomcat -p 8081:8080 docker-multistage:v1
docker ps
```

Screenshot evidence: Running container and PORTS 0.0.0.0:8081->8080/tcp

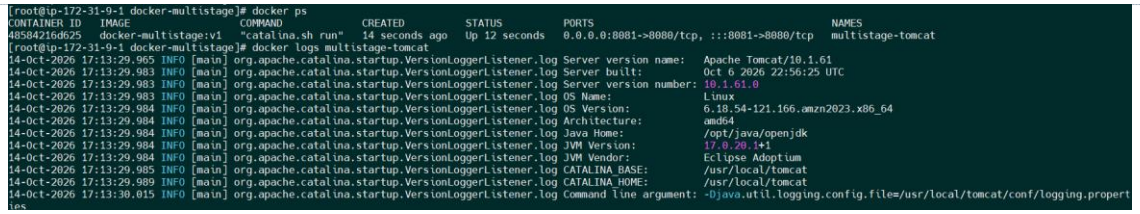


Step 9: Verify application in browser

Open http://localhost:8081/docker-demo/ in the browser. Expected text: Docker Multi-Stage Build Successful! If Docker is on a remote EC2 instance, use its reachable IP and port 8081 instead, and allow the port in its security group.

```
docker logs multistage-tomcat
```

Screenshot evidence: Browser output and optional Tomcat logs

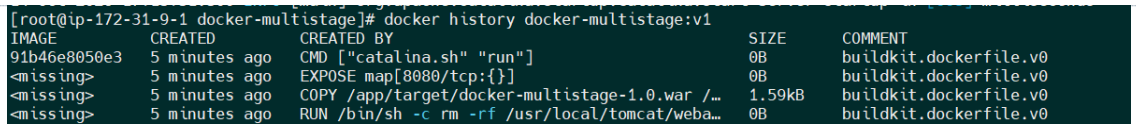


Step 10: Inspect image history

View image layers and check that Maven build tools are not part of the final runtime stage:

```
docker history docker-multistage:v1
docker image inspect docker-multistage:v1
```

Screenshot evidence: Image history or inspect output



Step 11: Optional: Compare with a single-stage image

A single-stage Maven image would include Maven/build dependencies at runtime if used directly. Multi-stage builds keep build dependencies out of the final Tomcat image. For a fair numerical size comparison, build and measure a working single-stage alternative separately.

```
docker image ls docker-multistage
```

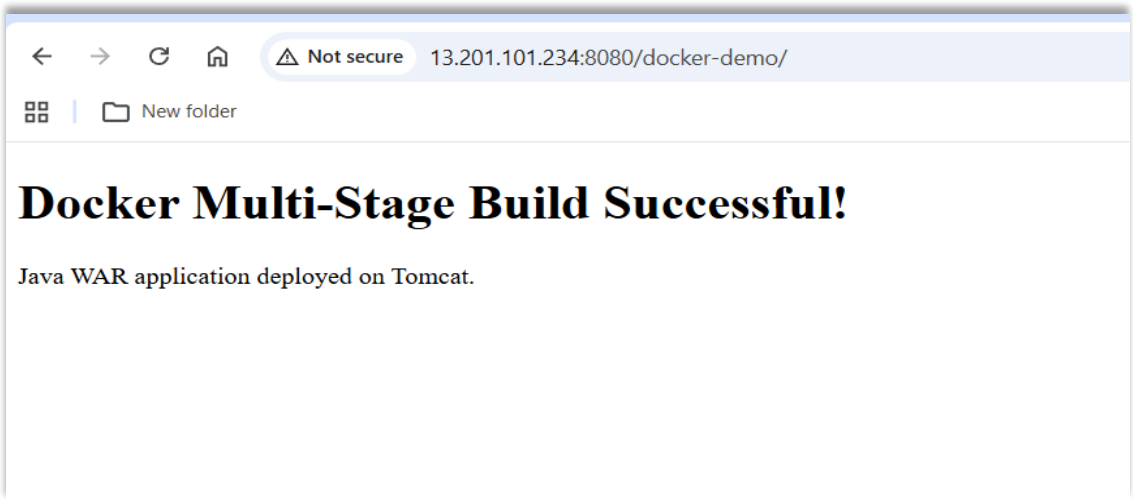
Screenshot evidence: Final image size



Your Docker image and containers are running successfully! In your screenshot, I can see that docker-multistage:v1 was built and that two containers are running:

```
[root@ip-172-31-9-1 docker-multistage]# docker image ls docker-multistage
REPOSITORY          TAG       IMAGE ID       CREATED          SIZE
docker-multistage   v1        91b46e8050e3   7 minutes ago   478MB
[root@ip-172-31-9-1 docker-multistage]# docker images
REPOSITORY          TAG       IMAGE ID       CREATED          SIZE
docker-multistage   v1        91b46e8050e3   11 minutes ago   478MB
[root@ip-172-31-9-1 docker-multistage]# docker run -d --name multistage-app -p 8080:8080 docker-multistage:v1
48fcd195bc21f9c2290639e1c8cf59ce615f1a2140eced6497784d8dfbf0e853
[root@ip-172-31-9-1 docker-multistage]# docker ps
CONTAINER ID   IMAGE                 COMMAND             CREATED         STATUS         PORTS                                                NAMES
48fcd195bc21   docker-multistage:v1   "catalina.sh run"   8 seconds ago   Up 7 seconds   0.0.0.0:8080->8080/tcp, :::8080->8080/tcp   multistage-app
48584216d625   docker-multistage:v1   "catalina.sh run"   8 minutes ago   Up 8 minutes   0.0.0.0:8081->8080/tcp, :::8081->8080/tcp   multistage-tomcat
[root@ip-172-31-9-1 docker-multistage]#
```

Step 12: Browser test and final image verification before cleanup.



Step 13: Cleanup after verification

Remove the test container and image only after taking screenshots:

```
docker stop multistage-tomcat
docker rm multistage-tomcat
docker rmi docker-multistage:v1
```

Screenshot evidence: Cleanup commands (optional)

```
[root@ip-172-31-9-1 docker-multistage]#
[root@ip-172-31-9-1 docker-multistage]# docker ps -a
CONTAINER ID   IMAGE                 COMMAND             CREATED         STATUS         PORTS                                                NAMES
48fcd195bc21   docker-multistage:v1   "catalina.sh run"   9 minutes ago   Up 9 minutes   0.0.0.0:8080->8080/tcp, :::8080->8080/tcp   multistage-app
[root@ip-172-31-9-1 docker-multistage]# docker rm -f multistage-app
multistage-app
[root@ip-172-31-9-1 docker-multistage]# docker rmi docker-multistage:v1
Untagged: docker-multistage:v1
Deleted: sha256:91b46e8050e3a8d2e561fa3480d3d20ba484360a71c403aed04accf36bfaad9f
[root@ip-172-31-9-1 docker-multistage]# docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
[root@ip-172-31-9-1 docker-multistage]#
```

Architecture / Build Flow

Source files + pom.xml → Maven builder stage → docker-demo.war → Tomcat runtime stage → Container port 8080 → Host port 8081 → Browser